

Module 35: Testing & Debugging OO Code

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

This module teaches students to prove their classes work and to find out quickly why they don't. They begin with `assert`, then learn unit testing with Arrange-Act-Assert using a 40-line `minitest.h` framework that shows there is no magic in real frameworks. They decide what to test in a class (boundaries, exceptions, state after failure, invariants), test in isolation with fakes made possible by dependency injection, and practise test-driven development. They then use Catch2 with CMake and `ctest`, debug with `gdb`, and catch memory errors with `AddressSanitizer` and `UBSan`. Each project starter hides exactly one bug for the students' tests to find.

Suggested Duration: 90–105 minutes

Learning Objectives

By the end of this module, students will be able to:

- Explain why automated tests matter, and use `assert` for internal assumptions.
- Write unit tests with Arrange-Act-Assert, one behaviour per test, with descriptive names.
- Decide what to test in a class: normal cases, boundaries, invalid input, exceptions, and state after failure.
- Test classes in isolation using fakes injected through interfaces.
- Practise test-driven development with red-green-refactor cycles.
- Build and run Catch2 tests with CMake and `ctest`.
- Debug with `gdb` (or an IDE debugger) and find memory errors with sanitizers.

Prerequisites

- Module 33 (dependency inversion and fakes).
- Module 22 (CMake) and Module 31 (exceptions and guarantees).

Materials & Instructor Prep

- 35-testing-debugging/examples/minitest.h and examples 01 to 08.
- examples/08_catch2_project needs internet on its first build to download Catch2; build it once before the session.
- `gdb` installed (or `lldb` on macOS, or VS Code's debugger).

Key Vocabulary

Term	Definition
Unit test	A small automated check of one behaviour of one unit (usually a class or function) in isolation.
Arrange-Act-Assert	The three-part shape of a test: set up objects, perform one action, check the result.
Fake	A stand-in implementation of an interface that records calls or returns controlled values, used in tests.
Test-driven development (TDD)	Writing a failing test first (red), the simplest code to pass it (green), then improving the code (refactor).
Catch2	A popular C++ unit-testing framework using TEST_CASE, REQUIRE, CHECK and SECTION.
Debugger (gdb)	A tool to pause a running program, step through it, and inspect variables and the call stack.
Sanitizer	Compiler-inserted checks (AddressSanitizer, UBSan) that detect memory errors and undefined behaviour as they happen.

Lesson Outline & Instructor Talking Points

1. Why test? assert and Arrange-Act-Assert (15 min)

- Ask: how did you know your Module 33 refactoring didn't break anything? (Diffing output.) Present tests as the scalable version of that.
- Run examples/01_assert_basics.cpp, then examples/02_first_unit_tests.cpp, reading the deliberate FAIL line together.
- Open minitest.h and show the macros capturing #cond and __LINE__.

2. What to test, and testing with fakes (25 min)

- Run examples/03_testing_exceptions.cpp; point at the strong-guarantee checks from Module 31.
- Run examples/04_fake_dependency.cpp. Ask how you would test the three-in-a-row alarm rule with a real thermometer (you can't, reliably).

Instructor note: *This is where Module 33's dependency inversion pays off visibly. Say it explicitly: code that creates its own dependencies can't be tested this way.*

3. TDD (15 min)

- Walk through examples/05_tdd_walkthrough.cpp cycle by cycle, reading the comments as a story.
- Optionally live-code one further cycle (e.g. 'a slot longer than 4 hours is rejected') with the class.

4. Catch2 and ctest (15 min)

- Show examples/08_catch2_project: two executables sharing src/, Catch2 via FetchContent, and ctest. Explain REQUIRE vs CHECK and SECTION.

5. Debugging and sanitizers (20 min)

- Debug examples/07_debugger_demo.cpp live in gdb using the walkthrough in its comments, until the class spots the loop starting at 1.
- Build examples/06_sanitizer_demo.cpp with -fsanitize=address,undefined and run each mode; read one report top to bottom.

Instructor note: *Running the same buggy program with and without sanitizers - silent nonsense vs a precise report - is the most convincing argument for always testing with sanitizers on.*

Common Student Misconceptions

Misconception	How to Address It
"Testing means checking the program once by hand before submitting."	Automated tests are re-run after every change, in seconds, and catch regressions a manual check misses.
"A test should test as much as possible at once."	One behaviour per test makes failures point straight at the problem and keeps tests independent.
"If the program doesn't crash, there's no memory bug."	Use-after-free and overflows often produce wrong values silently. Sanitizers reveal them immediately.
"Tests should check private details to be thorough."	Test public behaviour. Tests tied to private details break on every refactor and stop being a safety net.

Exercises & Module Project

Exercise: 35-testing-debugging/exercises/exercise1.cpp - write a minitest suite from a written specification for a TemperatureLog class, let the tests reveal its one bug, fix it, and add a test for an unspecified behaviour. Project (three tracks): Generic - FeeAccount and FeeReminder; EE - Circuit and LoadMonitor; Biomedical - DoseCalculator and PrescriptionChecker. Each starter hides exactly one bug; students write 25+ checks with fakes, commit the failing test before the fix, TDD a new feature, demonstrate a sanitizer report, and optionally port tests to Catch2.

Facilitation guidance: Check the commit history: the failing test should be committed before the fix. Read TESTING.md for the TDD cycle log and the sanitizer evidence. A good suite finds the hidden bug through a boundary test - ask students which test found it and why the original author missed it.

Discussion & Comprehension Check Questions

1. Why should unit tests avoid real files, networks, keyboards and clocks?

Guidance: They make tests slow, fragile and unrepeatable. Fakes give controlled, instant, identical behaviour every run.

2. What does writing the test first in TDD give you?

Guidance: You design the interface from the caller's point of view, and no code exists without a test that asked for it.

3. When a bug is found, what should you do before fixing it?

Guidance: Write a test that reproduces it and watch it fail; after the fix it passes and the bug can't silently return.

Assessment Ideas

- Students write Arrange-Act-Assert tests for a given small class, including at least two boundaries and one exception.
- Give a sanitizer report and ask students to explain what went wrong and where, from the report alone.

Answer Key Notes

See 35-testing-debugging/exercises/solutions/exercise1_solution.cpp - the bug is the `averageOfLast()` loop skipping the most recent reading.

Wrap-Up & Bridge to Next Module

Bridge forward: "You can now design, build and prove a system. The final module puts all of Part 3 together: Capstone Project 3."

Instructor Tips

- If time is short, protect fakes (step 2) and sanitizers (step 5); Catch2 can be demonstrated from a pre-built project.
- Build the Catch2 example before class - the first build downloads Catch2 and takes a few minutes.